```
# empty tuple
my_tuple = ()

# tuple having integers
my_tuple = (1, 2, 3)

# tuple with mixed datatypes
my_tuple = (1, "Hello", 3.4)

# nested tuple
my_tuple = ("mouse", [8, 4, 6], (1, 2, 3))

# tuple can be created without parentheses
# also called tuple packing
my_tuple = 3, 4.6, "dog"
# tuple unpacking is also possible
a, b, c = my_tuple
```

```
# 1. This creates a string, not a tuple
my_tuple_1 = ("hello")  # only parentheses is not enough
print(type(my_tuple_1))  # Output: <class 'str'>

# 2. This correctly creates a single-item tuple
my_tuple_2 = ("hello",) # need a comma at the end
print(type(my_tuple_2))  # Output: <class 'tuple'>

# 3. This also correctly creates a single-item tuple
my_tuple_3 = "hello",   # parentheses is optional
print(type(my_tuple_3))  # Output: <class 'tuple'>
```

```
<class 'str'>
<class 'tuple'>
<class 'tuple'>
```

```
# 1. Defining a list (named 'my_tuple' in the example image)
my_tuple = ['p', 'e', 'r', 'm', 'i', 't']

print(my_tuple[0])  # Output: 'p'
print(my_tuple[5])  # Output: 't'

# Demonstrating an IndexError
try:
    print(my_tuple[6])  # # index must be in range
except IndexError as e:
    print(f"IndexError: {e}")

# Demonstrating a TypeError
try:
    print(my_tuple[2.0])  # # index must be an integer
except TypeError as e:
    print(f"TypeError: {e}")


# 2. Defining a nested tuple
n_tuple = ("mouse", [8, 4, 6], (1, 2, 3))

print(n_tuple[0][3])  # # nested index -> 's' (4th letter of "mouse")
print(n_tuple[1][1])  # # nested index -> 4 (2nd item of the list)
print(n_tuple[2][0])  # # nested index -> 1 (1st item of the nested tuple)
```

```
p
t
IndexError: list index out of range
TypeError: list indices must be integers or slices, not float
s
4
1
```

```
# Defining the list
my_tuple = ['p', 'e', 'r', 'm', 'i', 't']

# Negative indexing starts from the end of the list
print(my_tuple[-1])  # Output: 't' (the last item)
print(my_tuple[-6])  # Output: 'p' (the first item, counting backwards)
```

```
t
p
```

```
# Defining the tuple
my_tuple = ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')

# Slicing syntax: [start:stop] -> includes start, excludes stop
print(my_tuple[1:4])  # Output: ('r', 'o', 'g') (elements 2nd to 4th)
print(my_tuple[:-7])  # Output: ('p', 'r')       (elements beginning to 2nd)
print(my_tuple[7:])   # Output: ('i', 'z')       (elements 8th to end)
print(my_tuple[:])    # Output: ('p', 'r', ..., 'z') (elements beginning to end)
```

```
('r', 'o', 'g')
('p', 'r')
('i', 'z')
('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
```

```
# 1. Tuples themselves are immutable (cannot be changed)
my_tuple = (4, 2, 3, [6, 5])

# This will fail: Cannot change a direct integer element
try:
    my_tuple[1] = 9
except TypeError as e:
    print(f"TypeError: {e}")  # 'tuple' object does not support item assignment

# This will also fail: Cannot replace the list object itself
try:
    my_tuple[3] = 9
except TypeError as e:
    print(f"TypeError: {e}")

# This succeeds: You can modify a mutable object (like a list) inside a tuple
my_tuple[3][0] = 9
print(my_tuple)  # Output: (4, 2, 3, [9, 5])


# 2. Reassigning the variable name to a brand new tuple is completely fine
my_tuple = ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
print(my_tuple)  # Output: ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
```

```
TypeError: 'tuple' object does not support item assignment
TypeError: 'tuple' object does not support item assignment
(4, 2, 3, [9, 5])
('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
```

```
# Defining the tuple
my_tuple = ('a', 'p', 'p', 'l', 'e')

# Membership test using 'in' and 'not in'
print('a' in my_tuple)      # Output: True
print('b' in my_tuple)      # Output: False
print('g' not in my_tuple)  # Output: True
```

```
True
False
True
```

```
# Iterating through a tuple using a for loop
for name in ('John', 'Kate'):
    print("Hello", name)
```

```
Hello John
Hello Kate
```